

Equazione del Calore 2D e Riordinamento nelle Fattorizzazioni Cholesky

Giosuè Aiello

1 Impostazione del problema

L'obiettivo del progetto è la risoluzione numerica dell'equazione del calore in regime stazionario in due dimensioni spaziali. Il dominio di calcolo è il quadrato unitario $\Omega = [0, 1]^2$, e si cerca la funzione di temperatura $u : \Omega \rightarrow \mathbb{R}$ che soddisfa l'equazione ellittica:

$$\kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0, \quad (x, y) \in \Omega, \quad (1)$$

con condizioni al contorno di Dirichlet omogenee sul bordo $\partial\Omega$:

$$u(x, y) = u_0(x, y) \equiv 0, \quad (x, y) \in \partial\Omega.$$

I parametri fisici del problema sono: coefficiente di diffusività termica $\kappa = 0.01$ e termine sorgente $f(x, y) = e^{-10(x^2+y^2)}$, che modella una sorgente di calore concentrata in prossimità dell'origine con decadimento gaussiano.

Discretizzazione alle differenze finite

Si introduce una griglia cartesiana uniforme di $(N + 2)^2$ punti (x_i, y_j) con $i, j = 0, \dots, N + 1$ e passo $h = 1/(N + 1)$, dove $x_i = i \cdot h$ e $y_j = j \cdot h$. I punti di bordo hanno temperatura fissata a zero e non contribuiscono come incognite. Le N^2 incognite del problema sono i valori $u_{i,j} \approx u(x_i, y_j)$ nei punti *interni* ($1 \leq i, j \leq N$).

Approssimando le derivate seconde con lo stencil a cinque punti, l'equazione (1) valutata nel punto interno (x_i, y_j) diventa:

$$\frac{\kappa}{h^2} (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}) + f(x_i, y_j) = 0. \quad (2)$$

Se un vicino cade sul bordo, il suo valore (zero) viene incorporato nel termine noto senza aggiungere incognite. L'insieme delle equazioni (2) costituisce il sistema lineare $Au = b$, dove $A \in \mathbb{R}^{N^2 \times N^2}$ è sparsa (al più 5 elementi non nulli per riga), simmetrica e definita negativa.

Fattorizzazione di Cholesky e fill-in

Poiché $-A$ è simmetrica definita positiva (SPD), si può calcolare la fattorizzazione di Cholesky $-A = LL^T$ con L triangolare inferiore sparsa. Tuttavia, durante la fattorizzazione si generano elementi non nulli in posizioni che erano zero in $-A$: questo fenomeno è detto *fill-in* e dipende fortemente dall'ordine con cui le incognite sono enumerate.

Con l'ordinamento naturale per righe, la larghezza di banda di A è $\Theta(N)$ e il fill-in di L è $O(N^3)$. La tecnica di *nested dissection* riduce il fill-in a $O(N^2 \log N)$, con un risparmio sostanziale per valori grandi di N .

Grafo di adiacenza e nested dissection

La struttura di A è codificata dal *grafo di adiacenza* $G = (V, E)$: ogni nodo $v \in V$ corrisponde a una variabile $u_{i,j}$ e ogni arco $(u, v) \in E$ indica che $A_{uv} \neq 0$. Per la griglia regolare, due nodi sono collegati se e solo se sono vicini orizzontalmente o verticalmente:

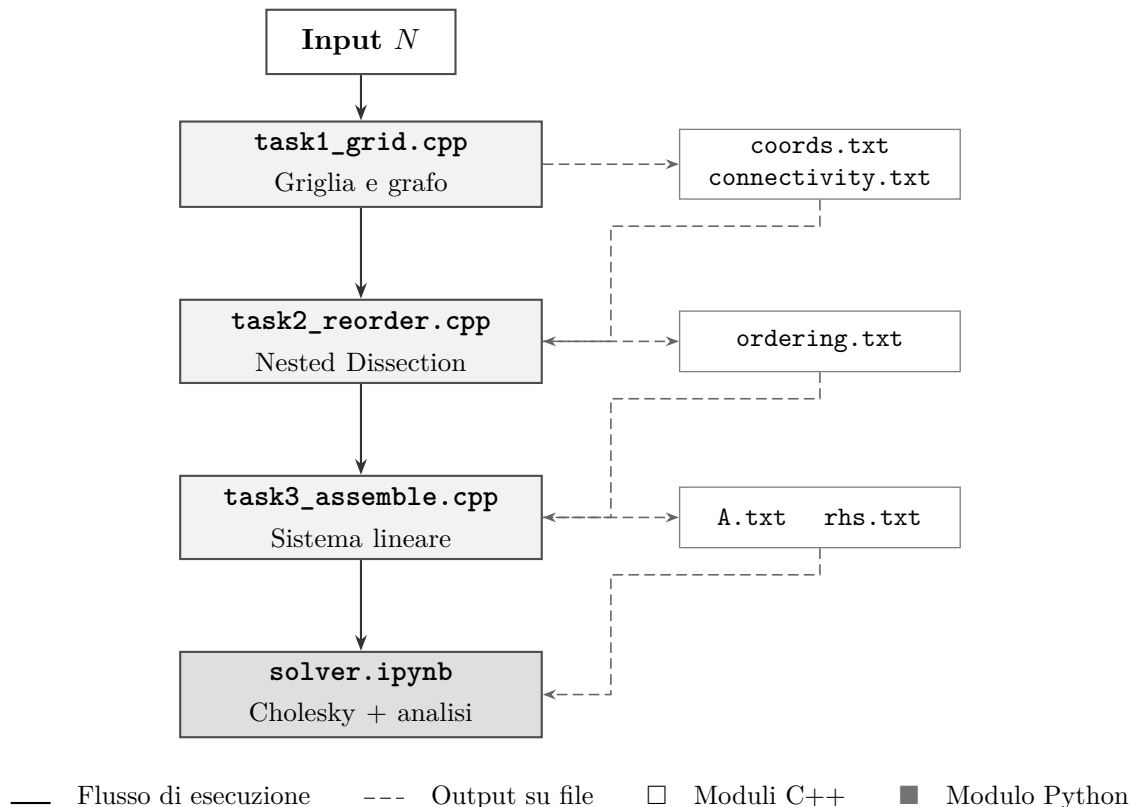
$$(x_i, y_j) \leftrightarrow (x_k, y_l) \iff (i = k, |j - l| = 1) \text{ oppure } (j = l, |i - k| = 1).$$

La *nested dissection* individua ricorsivamente un separatore geometrico V_S di cardinalità ridotta che divide V in due insiemi bilanciati V_1 e V_2 senza archi diretti tra loro. L'ordinamento risultante elenca prima V_1 , poi V_2 , infine V_S , in modo che il separatore appaia in fondo alla matrice, limitando la propagazione del fill-in. Per la griglia regolare il separatore ottimale è la colonna mediana: scelto $\hat{x}$ mediano, si pone $V_1 = \{u_{i,j} \mid x_i < \hat{x}\}$, $V_2 = \{u_{i,j} \mid x_i > \hat{x}\}$, $V_S = \{u_{i,j} \mid x_i = \hat{x}\}$. Il procedimento viene applicato ricorsivamente a V_1 e V_2 , alternando il taglio lungo x e y .

2 Architettura del progetto

Il progetto è strutturato in quattro moduli. I tre moduli in C++ comunicano tra loro attraverso file di testo con formato fisso che fungono da interfaccia verificabile indipendentemente; il modulo Python li legge come punto di ingresso per la fase numerica finale.

Il diagramma seguente illustra il flusso di elaborazione dall'input N alla soluzione e all'analisi comparativa:



Ogni modulo ha un ruolo preciso all'interno della pipeline:

task1_grid.cpp

C++

Riceve N ; genera i N^2 nodi interni con indici e coordinate (`coords.txt`) e la lista degli archi del grafo di adiacenza (`connectivity.txt`).

task2_reorder.cpp

C++

Legge il grafo e calcola la permutazione nested dissection ricorsiva: ad ogni livello individua il separatore geometrico mediano (alternando asse x e y) e lo posiziona in coda. Salva la corrispondenza in `ordering.txt`.

task3_assemble.cpp

C++

Assembla la matrice sparsa A (stencil a 5 punti: $-4\kappa/h^2$ in diagonale, $+\kappa/h^2$ fuori diagonale) e il termine noto b . Supporta ordinamento naturale e nested dissection. Produce `A.txt` e `rhs.txt`.

solver.ipynb

Python

Carica `A.txt` e `rhs.txt` in formato CSC; fattorizza $-A = LL^T$ con `sksparse.cholmod`; risolve con `spsolve_triangular`. Confronta tempi per $N = 32 \dots 1024$; produce spy plot e grafici della soluzione $u(x, y)$.

Le strutture dati fondamentali sono: liste di adiacenza (`std::vector<std::vector<int>>`) per il grafo nei moduli C++, e matrici sparse in formato CSC (`scipy.sparse.csc_matrix`) nel modulo Python. Questo schema iniziale verrà progressivamente arricchito con la specifica di input, output, invarianti e complessità di ciascun modulo, man mano che il codice viene sviluppato.